PrismView 수집기 · 설계 결정 정리노트

2026. 10. 27 (화) · 첫 Java 프로젝트 · OKX USDT 무기한 선물 BTC·ETH 1분봉 · 범위: 과거 백필 + 주기 업데이트 + 누락 검사 + 조회 API (실시간 WebSocket 제외)

구현을 시작하기 전에 내려야 할 결정 12가지를 정리했다. 각 장은 **개념 → 선택지 비교 → 이 프로젝트 기준 판단 → 기본안** 순서이고, 중간중간 생각해 보기와 확인 문제가 있다. 기본안은 정답이 아니라 "이 조건이면 이것"이다. 조건이 다르면 결정도 달라져야 한다.

★ 필수 — 결정이 프로젝트 전체에 영향 · ☆ 가볍게 — 기본안으로 시작해도 됨 · 확인 문제는 아래 해설을 손으로 가리고 먼저 푼다 · **[문서 확인]** 표시는 구현 직전 공식 문서로 수치를 다시 확인할 것 · 시간이 없으면 결정 지도, 1·2·3·5·7장, 함정 모음만 본다

결정 지도 한눈에 — 이 노트의 기본안

데이터 규모	2종목 × 1분봉 ≈ 연 105만 행, 대략 0.1~0.2GB/년 (추정) → "작은 데이터"
DBMS	MariaDB (이미 아는 DB로 Java 학습에 집중) · 대안 PostgreSQL
스키마	1분봉 테이블 하나, PK (inst_id, ts), 가격·거래량 DECIMAL, 시간 BIGINT(UTC epoch ms)
DB 접근	JdbcTemplate + 배치 upsert (JPA는 조회 API 쪽에서 선택적으로)
데이터 소스	OKX `history-candles`, 요청당 100개, `after` 로 과거 방향 페이지네이션
실행 방식	Spring `@Scheduled(fixedDelay)` + "마지막 저장 ts 이후" 증분
정합성	LAG 윈도 함수로 공백 탐지 → 재요청 → 그래도 비면 known_gaps 기록
상위 시간봉	조회 시 SQL 즉석 집계, UTC 경계, 구성 봉 개수 검사
기반	Spring Boot · Gradle · Java 21 이상 LTS · Flyway · JUnit 5 + Testcontainers

1. 데이터의 성격부터 ★ 모든 결정의 기준

DB, 자료형, 집계 방식은 전부 "어떤 데이터가 얼마나, 어떻게 쓰이고 읽히는가"에서 나온다. 이 장의 숫자를 먼저 손에 쥐고 다음 장으로 넘어간다.

★ 캔들 한 개에 들어 있는 것

ts	봉의 **시작** 시각. OKX는 Unix 밀리초(UTC 기준) 문자열로 준다. 10:00 봉 = 10:00:00.000~10:00:59.999 구간
o · h · l · c	시가 · 고가 · 저가 · 종가. 문자열로 오므로 숫자 변환 필요
vol	거래량. **무기한 선물(SWAP)은 계약 수** 단위
volCcy	거래량을 기초 자산(BTC, ETH) 수량으로 환산한 값
volCcyQuote	거래량을 호가 통화(USDT) 금액으로 환산한 값
confirm	`"0"` = 아직 진행 중인 봉(값이 계속 바뀜) · `"1"` = 마감된 봉

계약 1개가 BTC 몇 개인지(계약 단위, ctVal)는 `GET /api/v5/public/instruments`로 확인한다. 하드코딩하지 않는다. **[문서 확인]**

> **생각해 보기 백테스트에 confirm = 0인 봉이 섞이면 무슨 일이 생길까?**
>
> 먼저 답을 생각해 보고 아래로 내려간다.
>
> 진행 중인 봉의 고가·저가·종가는 마감 전까지 계속 바뀐다. 그 값으로 시그널을 계산하면 "그 시점엔 알 수 없던 값"을 쓰게 되어 과거 결과가 실제보다 좋아 보이는 리페인팅이 생긴다. PrismView의 "마감된 봉만 사용" 원칙과 직결되므로, **1차 범위에서는 확정 봉만 저장**하는 것이 가장 단순하고 안전하다.

★ 규모 추정 — 직접 계산해 본다

> 1분봉 1년 = 60 × 24 × 365 = **525,600개** (종목 1개)
> 2종목 × 1년 = **1,051,200행** · 3년 ≈ 315만 행 · 5년 ≈ 526만 행
> 행 하나 ≈ 키(종목명 ~16B + ts 8B) + 숫자 7개(DECIMAL 각 ~9~12B) + 저장 엔진 오버헤드 → 대략 120~150B로 잡으면
> 1년 ≈ 105만 × 130B ≈ **약 0.14GB** · 5년 ≈ 0.7GB (추정치, 실제는 DB·인덱스에 따라 다름)

결론: 노트북 한 대의 어떤 DB도 여유 있게 감당하는 **작은 데이터**다. "대용량 시계열 DB가 필요한가?"라는 질문에 숫자로 답할 수 있어야 한다.

★ 접근 패턴

쓰기 ① 백필	처음 한 번, 수십만~수백만 행을 몰아서 insert → **배치 insert 성능**이 중요
쓰기 ② 증분	하루 1회 등, 수백~수천 행 → 성능보다 **중복·누락 없이**가 중요
읽기 ① 범위 조회	"BTC, 3월 1일~3월 31일" → (종목, 시간) 순서 인덱스가 핵심
읽기 ② 집계	1분봉 → 1시간봉, 일봉 → GROUP BY
읽기 ③ 전 구간 스캔	백테스트가 수년치를 처음부터 끝까지 순서대로 읽음
수정 · 삭제	거의 없음. 한 번 확정된 봉은 바뀌지 않는다(append-mostly)

확인 1. BTC, ETH 두 종목의 1분봉을 3년치 저장하면 대략 몇 행인가?

1) 약 158만 행
2) 약 315만 행
3) 약 631만 행
4) 약 1억 9천만 행

먼저 답을 고르고 내려간다.

정답 2) 525,600 × 3년 × 2종목 = 3,153,600. 1)은 종목 하나만 센 값, 4)는 분이 아니라 초 단위로 센 값이다. 단위(분/초)와 종목 수를 빠뜨리는 실수가 가장 흔하다.

2. DBMS 선택 ★ 가장 비중 큰 결정

★ 먼저 알아야 할 개념 — 행 기반 vs 열 기반

행 기반 (Row-oriented)	한 행의 모든 컬럼을 붙여서 저장. 한 행을 통째로 넣고 꺼내기에 유리. MariaDB(InnoDB), PostgreSQL, SQLite
열 기반 (Column-oriented)	컬럼별로 따로 모아 저장. "종가만 100만 개 평균"처럼 일부 컬럼을 대량 집계할 때 읽는 양이 적고 압축이 잘 됨. ClickHouse, DuckDB, Parquet 파일
시계열 특화	시간 축 파티셔닝, 시간 버킷 함수, 자동 집계 등을 기본 제공. TimescaleDB(PostgreSQL 확장), QuestDB
임베디드 vs 서버형	임베디드는 앱 프로세스 안에서 파일 하나로 동작(SQLite, DuckDB). 서버형은 별도 프로세스로 띄우고 네트워크로 접속

생각해 보기 열 기반 DB가 "종가 평균"에는 빠른데 "봉 하나를 통째로 수정"에는 불리한 이유는?

먼저 답을 생각해 보고 아래로 내려간다.

평균은 종가 컬럼 파일만 읽으면 되니 디스크에서 읽는 양이 1/9 수준으로 준다. 반면 한 행을 수정하려면 컬럼 9개가 각각 다른 곳에 흩어져 있어 9군데를 고쳐야 하고, 압축된 블록을 풀었다 다시 써야 한다. 그래서 열 기반 DB는 대개 "덧붙이기는 빠르고, 수정·중복 제거는 나중에 몰아서" 처리한다.

★ 선택지 비교

선택지	형태	강점	약점 · 주의
MariaDB / MySQL	행 · 서버	이미 익숙함. Spring·JDBC 연동 자료 최다. `ON DUPLICATE KEY UPDATE`로 upsert 간단. 윈도 함수 지원	시간 버킷 함수 같은 시계열 편의 기능 없음(직접 SQL로)
PostgreSQL	행 · 서버	표준 SQL 충실, 기능 풍부. `ON CONFLICT DO UPDATE`. 나중에 TimescaleDB로 확장 가능	새로 배워야 할 것이 생김(첫 Java 프로젝트와 동시 학습)
TimescaleDB	PostgreSQL 확장	하이퍼테이블(시간 자동 분할), `time_bucket`, 연속 집계(continuous aggregate). PostgreSQL SQL·드라이버 그대로	이 규모에선 기능 대부분이 과함. 일부 기능의 라이선스 조건 **[문서 확인]**
ClickHouse	열 · 서버	수십억 행 집계에 매우 빠름	upsert가 자연스럽지 않음. ReplacingMergeTree의 중복 제거는 백그라운드 병합 때 일어나 조회 시 중복이 보일 수 있음. 운영 개념이 다름
QuestDB	시계열 · 서버	지정 타임스탬프, `SAMPLE BY` 집계 문법, 빠른 수집	자료·커뮤니티가 상대적으로 적음. 중복 제거 설정 방식 **[문서 확인]**
SQLite	행 · 임베디드	설치·운영 0. 파일 하나	동시 쓰기 제한. "서버 DB를 다뤄봤다"는 포트폴리오 가치는 낮음
DuckDB	열 · 임베디드	분석·집계에 강함, Parquet 직접 조회	여러 프로세스가 동시에 쓰는 서버형 용도가 아님. 수집 저장소보다 분석 도구에 가까움
Parquet / CSV 파일	파일	백테스터(특히 Python)가 바로 읽기 좋음. 압축 효율 높음(Parquet)	upsert·누락 검사·동시 접근을 직접 구현해야 함

★ 이 프로젝트 기준 판단

1장 숫자대로라면 **성능 차이는 결정 기준이 되지 않는다.** 100만~500만 행 범위 조회·집계는 어떤 선택지든 체감 차이가 거의 없다. 그렇다면 기준은 이 셋이다.

① 학습 집중	이번 목표는 "Java/Spring 백엔드". 새 DB까지 동시에 배우면 막혔을 때 원인이 Java인지 DB인지 구분하기 어렵다
② 정합성 도구	upsert, PK 제약, 윈도 함수(누락 검사)가 쉽게 되는가 → 행 기반 RDB가 가장 자연스럽다
③ 설명 가능성	면접에서 "왜 이 DB?"에 1장 숫자로 답할 수 있는가. "유행이라서 시계열 DB"는 오히려 감점 요인

기본안 — MariaDB

아는 DB로 위험을 줄이고, 배울 에너지는 Java·Spring·JDBC 배치·정합성 설계에 쓴다. 금융 IT 현장에서도 RDB의 PK·트랜잭션·인덱스 설계 역량이 기본이다.

다른 선택이 나은 조건 · Java보다 DB 쪽 학습이 목표다 → PostgreSQL · 종목이 수백 개, 틱 데이터까지 간다 → TimescaleDB/ClickHouse 검토 · 백테스트를 Python에서만 한다 → DB는 그대로 두고 Parquet 내보내기 추가(10장)

확인 2. 이 프로젝트 규모에서 DBMS 선택 기준으로 가장 비중이 낮은 것은?

1) 공개 벤치마크의 초당 insert 처리량 수치
2) 내가 이미 다뤄 본 정도
3) upsert·PK 제약·윈도 함수 지원 여부
4) Spring/JDBC 연동 자료의 풍부함

먼저 답을 고르고 내려간다.

정답 1) 백필 전체가 100만 행 수준이라 어떤 DB든 몇 분 안에 끝난다. 벤치마크는 수억 행 이상일 때 의미가 있다. "빠른 DB"보다 "내 문제에 맞는 DB"를 고르는 습관이 이 장의 핵심이다.

확인 3. DBMS에 대한 설명으로 옳은 것은?

1) 행 기반 DB는 컬럼 몇 개만 대량 집계할 때 열 기반 DB보다 디스크에서 읽는 양이 적다.
2) ClickHouse의 ReplacingMergeTree는 insert 즉시 중복을 제거하므로 조회 시 항상 한 행만 보인다.
3) TimescaleDB는 PostgreSQL 확장이라 PostgreSQL의 SQL과 JDBC 드라이버를 그대로 쓸 수 있다.
4) DuckDB는 여러 애플리케이션이 동시에 접속해 쓰는 서버형 DB로 설계되었다.

먼저 답을 고르고 내려간다.

정답 3) 1) 반대다. 행 기반은 필요 없는 컬럼까지 같이 읽는다. 2) 중복 제거는 백그라운드 병합 시점에 일어나므로 그 전에는 중복이 보일 수 있다(조회 시 FINAL 등으로 보정). 4) DuckDB는 임베디드 분석 DB다.

3. 스키마 설계 ★ 바꾸기 가장 어려운 결정

★ 기본키 — 대리키 vs 자연키

대리키 (AUTO_INCREMENT id)	의미 없는 일련번호. 같은 봉을 두 번 넣으면 id만 다른 **중복 행 2개**가 생긴다. 막으려면 별도 UNIQUE 제약이 또 필요
자연키 (inst_id, ts)	"어떤 종목의 몇 시 봉"이 곧 식별자. DB가 중복을 원천 차단. InnoDB에서는 PK 순서로 데이터가 저장되어 **종목+시간 범위 조회가 PK 인덱스를 그대로 탄다**
멱등성 (Idempotency)	같은 작업을 여러 번 해도 결과가 같은 성질. 재시도·재실행·백필 구간 겹침이 일상인 수집기의 핵심 성질

확인 4. 캔들 테이블의 PK를 (inst_id, ts)로 잡는 가장 큰 이유는?

1) AUTO_INCREMENT보다 insert 속도가 항상 빠르기 때문에
2) 같은 봉을 여러 번 넣어도 한 행만 남아 재시도·재실행이 안전해지기 때문에
3) 테이블 저장 용량이 절반 이하로 줄기 때문에
4) JPA가 복합키를 가장 잘 지원하기 때문에

먼저 답을 고르고 내려간다.

정답 2) 수집기는 네트워크 오류 재시도, 노트북 재시작, 백필 겹침 때문에 같은 데이터를 반드시 여러 번 받는다. 자연키 + upsert면 그때마다 결과가 같다. 4)는 오히려 반대로, JPA에서 복합키는 `@IdClass` / `@EmbeddedId` 가 필요해 다소 번거롭다(4장).

★ 가격·거래량 자료형

DOUBLE (부동소수점)	2진수로 근사 저장. 0.1 + 0.2 = 0.30000000000000004. 합계·비교에서 오차가 누적. **금융 데이터에 쓰지 않는다**
DECIMAL(p, s)	10진수 그대로 정확히 저장. Java에서는 `BigDecimal` 로 매핑. 기본안

BIGINT 스케일링	가격 × 10^8을 정수로 저장. 가장 빠르고 작지만, 읽고 쓸 때마다 변환 규칙을 지켜야 하고 실수 위험이 큼. 대규모 거래 시스템에서 쓰는 방식

> **생각해 보기 DOUBLE로 저장한 종가로 "종가가 정확히 65000.1일 때 진입" 조건을 검사하면?**
>
> 먼저 답을 생각해 보고 아래로 내려간다.
>
> ---
>
> 65000.1은 2진수로 정확히 표현되지 않아 65000.09999… 같은 값으로 저장될 수 있고, `==` 비교가 실패할 수 있다. 백테스트에서 조건이 "가끔" 안 맞는 버그는 찾기 매우 어렵다. Java에서도 `BigDecimal` 끼리는 `equals`가 아니라 `compareTo`로 비교해야 한다 (`equals`는 1.0과 1.00을 다르게 본다).

★ 시간 자료형

BIGINT (epoch ms)	OKX가 주는 값 그대로. 타임존 변환이 끼어들 여지가 없음. 버킷 계산이 정수 나눗셈 한 번. 기본안
DATETIME	타임존 정보가 없는 "벽시계 값". 넣는 쪽이 UTC로 넣었는지 KST로 넣었는지 테이블만 봐서는 모름
TIMESTAMP	MariaDB/MySQL에서 세션 타임존 기준으로 변환되어 저장·조회됨. 서버·JVM·세션 타임존이 어긋나면 9시간 밀린 데이터가 생김. 지원 범위도 버전별 제한 **[문서 확인]**

사람이 읽을 때는 조회 SQL에서 `FROM_UNIXTIME(ts/1000)`으로 변환하거나 API 응답 단계에서 ISO-8601 문자열로 바꾼다. 저장은 정수, 표시는 변환.

★ 무엇을 저장할 것인가

1분봉만 저장	원본 하나에서 모든 시간봉을 만든다. 원본이 하나라 정합성 관리가 쉬움. 기본안
시간봉별 테이블	OKX에서 1H, 1D를 따로 받아 저장. 조회는 빠르지만 원본이 여러 개 → 1분봉 합과 1시간봉이 어긋나면 어느 쪽이 맞는지 판단 불가
확정 봉만 저장	1장 결론. `confirmed` 컬럼은 나중에 실시간 수집을 붙일 때를 대비해 넣어 둘 수 있음(1차는 항상 1)
인덱스 · 파티셔닝	PK(inst_id, ts) 하나로 범위 조회가 해결됨. 파티셔닝은 수억 행 이상, 오래된 데이터를 통째로 지울 일이 생길 때 검토. 지금은 불필요

직접 해보기 — MariaDB DDL과 upsert

```
CREATE TABLE candle_1m (
  inst_id    VARCHAR(20)    NOT NULL,            -- 'BTC-USDT-SWAP'
  ts         BIGINT         NOT NULL,            -- 봉 시작, UTC epoch ms
  open       DECIMAL(18,8)  NOT NULL,
  high       DECIMAL(18,8)  NOT NULL,
  low        DECIMAL(18,8)  NOT NULL,
  close      DECIMAL(18,8)  NOT NULL,
  vol        DECIMAL(24,8)  NOT NULL,            -- 계약 수
  vol_ccy    DECIMAL(24,8)  NOT NULL,            -- BTC/ETH 수량
  vol_quote  DECIMAL(24,8)  NOT NULL,            -- USDT 금액
  confirmed  TINYINT(1)     NOT NULL DEFAULT 1,
  PRIMARY KEY (inst_id, ts)
) ENGINE=InnoDB;

INSERT INTO candle_1m (inst_id, ts, open, high, low, close, vol, vol_ccy, vol_quote, confirmed)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
ON DUPLICATE KEY UPDATE
  open = VALUES(open), high = VALUES(high), low = VALUES(low), close = VALUES(close),
  vol = VALUES(vol), vol_ccy = VALUES(vol_ccy), vol_quote = VALUES(vol_quote),
  confirmed = VALUES(confirmed);
```

① 같은 행을 두 번 실행하고 `COUNT(*)` 가 1인지 확인한다. ② `DECIMAL(18,8)` 이면 정수부가 몇 자리까지 들어가는지 계산해 본다 (18 − 8 = 10자리).

확인 5. PostgreSQL의 upsert `INSERT … ON CONFLICT (inst_id, ts) DO UPDATE SET close = ___.close` 에서 빈칸, 즉 "새로 넣으려던 값"을 가리키는 키워드는?

1) VALUES
2) EXCLUDED
3) NEW
4) INSERTED

먼저 답을 고르고 내려간다.

정답 2) PostgreSQL은 `EXCLUDED.close` . MariaDB는 `VALUES(close)` (MySQL 8.0.20 이후는 행 별칭 방식을 권장). `NEW` 는 트리거, `INSERTED` 는 SQL Server 문법이다. DB를 바꾸면 upsert SQL이 달라진다는 점이 4장 "DB 접근 방식" 선택과 연결된다.

4. Java에서 DB에 접근하는 방식 ★ 대량 insert에서 갈림

방식	성격	이 프로젝트에서
JPA / Hibernate	객체 ↔ 테이블 자동 매핑(ORM). 엔티티 상태 추적. SQLAlchemy ORM에 해당	대량 upsert에 불리. 복합키 설정이 번거로움. 조회 API 쪽 학습용으로는 가치 있음
JdbcTemplate	SQL을 직접 쓰고 실행·매핑만 도와줌. SQLAlchemy Core + text()에 해당	배치 upsert를 SQL 그대로 제어. 기본안
MyBatis	SQL을 XML/어노테이션에 두고 매핑. 국내 금융권 SI에서 많이 씀	가능. 다만 Spring Boot 기본 구성보다 설정이 하나 늘어남
jOOQ	DB 스키마에서 코드를 생성해 타입 안전한 SQL 작성	강력하지만 첫 프로젝트에는 학습량이 큼

★ JPA로 대량 insert가 느려지는 이유 3가지

IDENTITY 전략	AUTO_INCREMENT id는 insert 후에야 값이 정해지므로 Hibernate가 insert를 JDBC 배치로 묶지 못한다

save()의 SELECT	id를 직접 지정한 엔티티는 Spring Data JPA가 "새 엔티티인지"를 id만으로 판단하지 못해 merge를 호출 → insert 전에 SELECT가 나갈 수 있다 (`Persistable` 로 우회 가능)
영속성 컨텍스트	저장한 엔티티를 1차 캐시에 계속 쌓아 메모리 사용과 dirty checking 비용이 커진다 → 주기적 flush/clear 필요

> **생각해 보기 JPA `saveAll()` 로 1분봉 10만 개를 넣었더니 SQL 로그에 SELECT가 10만 번 찍혔다. 왜일까?**
>
> 먼저 답을 생각해 보고 아래로 내려간다.
>
> ---
>
> (inst_id, ts)처럼 키를 직접 채운 엔티티는 "id가 null이면 새 것"이라는 기본 판단이 안 통해서, 매 건마다 "이미 있는지" SELECT로 확인한 뒤 insert 또는 update를 한다. 의도는 upsert인데 DB 왕복이 두 배가 된다. 처음부터 SQL upsert + JDBC 배치로 쓰면 이 문제가 없다.

★ JDBC 배치 옵션은 드라이버마다 다르다

MySQL Connector/J	`rewriteBatchedStatements=true` 를 켜면 배치를 multi-values INSERT 하나로 다시 써서 빨라진다
MariaDB Connector/J 3.x	`rewriteBatchedStatements` 옵션이 **제거**되었다. 대신 MariaDB 서버의 bulk 프로토콜을 쓰며, 3.2부터 INSERT에 대한 bulk(`useBulkStmtsForInserts`)가 기본 활성화 **[문서 확인]**

> **확인 6. Java에서 대량 insert에 대한 설명으로 옳지 않은 것은?**
>
> 1) JPA에서 IDENTITY 전략을 쓰면 Hibernate는 insert를 JDBC 배치로 묶지 못한다.
> 2) 식별자를 직접 지정한 엔티티에 Spring Data JPA save()를 호출하면 insert 전에 SELECT가 나갈 수 있다.
> 3) MariaDB Connector/J 3.x에서도 rewriteBatchedStatements=true를 켜야 배치 insert가 빨라진다.
> 4) JdbcTemplate.batchUpdate는 여러 행을 하나의 배치로 묶어 보낼 수 있다.
>
> 먼저 답을 고르고 내려간다.
>
> ---
>
> **정답 3)** 그 옵션은 MySQL 드라이버 또는 MariaDB 드라이버 2.x 이하 이야기다. 3.x에서는 제거되었고 bulk 방식이 기본이다. 블로그 글을 옮길 때 **드라이버 이름과 버전**부터 확인하는 습관이 필요하다.

직접 해보기 — 도메인 레코드와 배치 upsert 뼈대

```
public record Candle(String instId, long ts,
                     BigDecimal open, BigDecimal high, BigDecimal low, BigDecimal close,
                     BigDecimal vol, BigDecimal volCcy, BigDecimal volQuote, boolean confirmed) {

    // OKX 응답 한 줄: ["ts","o","h","l","c","vol","volCcy","volCcyQuote","confirm"]
    public static Candle fromOkx(String instId, List<String> r) {
        return new Candle(instId, Long.parseLong(r.get(0)),
            new BigDecimal(r.get(1)), new BigDecimal(r.get(2)),
            new BigDecimal(r.get(3)), new BigDecimal(r.get(4)),
            new BigDecimal(r.get(5)), new BigDecimal(r.get(6)), new BigDecimal(r.get(7)),
            "1".equals(r.get(8)));
    }
}

jdbcTemplate.batchUpdate(UPSERT_SQL, candles, 1000, (ps, c) -> {
    ps.setString(1, c.instId());  ps.setLong(2, c.ts());
    ps.setBigDecimal(3, c.open()); /* … 나머지 컬럼 … */
});
```

`new BigDecimal("65000.1")` (문자열)과 `new BigDecimal(65000.1)` (double)을 각각 출력해 차이를 확인해 본다.

5. 데이터 가져오기 — OKX REST ★ 페이지네이션 방향 함정

★ 엔드포인트 두 개

엔드포인트	범위	요청당 개수	호출 제한
`/api/v5/market/candles`	최근 데이터만 (최신 1,440개까지)	최대 300, 기본 100	[문서 확인]
`/api/v5/market/history-candles`	최근 수년치 과거 데이터	최대 100	2초당 20회 (IP 기준)

after	이 ts보다 **이전(과거)** 데이터를 요청. 백필에 사용
before	이 ts보다 **이후(최신)** 데이터를 요청
응답 정렬	**최신순**(내림차순). 저장 전 정렬이 필요한지는 DB 쪽에선 상관없지만 커서 계산엔 중요
bar 값	1m, 3m, 5m, 15m, 30m, 1H, 2H, 4H … 그리고 6H, 12H, 1D, 1W 등은 **기본이 홍콩 시간 (UTC+8) 기준**, UTC 기준은 `1Dutc`처럼 접미사를 붙인다
가장 오래된 봉	상장일 이전은 데이터가 없고, 1분봉을 몇 년 전까지 주는지는 직접 요청해서 확인 [문서 확인]

생각해 보기 최신순 응답에서 다음 페이지 요청의 after 값은 무엇이어야 할까?

먼저 답을 생각해 보고 아래로 내려간다.

이번 응답의 **마지막 원소(가장 오래된 봉)의 ts**. 첫 원소를 쓰면 같은 페이지를 무한히 다시 받는다. 응답이 빈 배열이면 더 과거 데이터가 없다는 뜻이니 종료한다. 페이지가 겹쳐도 upsert라 안전하지만, 커서가 전진하지 않으면 레이트 리밋만 낭비된다 → "커서가 반드시 줄어드는지" 단위 테스트로 검증한다.

```
Long cursor = null;                                  // null → 가장 최근부터
while (true) {
    limiter.acquire();                               // 호출 제한 준수
    List<Candle> page = okx.historyCandles(instId, "1m", cursor, 100);  // 최신순
    if (page.isEmpty()) break;
    repo.upsertAll(page.stream().filter(Candle::confirmed).toList());
    long oldest = page.get(page.size() - 1).ts();
    if (cursor != null && oldest >= cursor) throw new IllegalStateException("커서 정지");
    cursor = oldest;
    if (cursor <= targetStartTs) break;              // 목표 시점까지 도달
}
```

확인 7. history-candles(요청당 100개, 2초당 20회)로 BTC·ETH 1분봉 1년치를 호출 제한 한도까지 꽉 채워 받으면 대략 얼마나 걸리나?

1) 약 2분
2) 약 18분
3) 약 35분
4) 약 3시간

먼저 답을 고르고 내려간다.

정답 2) 1,051,200행 ÷ 100 = 10,512회. 초당 10회 → 약 1,051초 ≈ 17.5분. 한도를 꽉 채우면 429(요청 과다)를 맞기 쉬우니 실제로는 80% 정도(초당 8회)로 잡아 20분 남짓을 예상한다. 3)은 초당 5회로 계산한 값이다.

확인 8. OKX에서 bar=1D(접미사 없음)로 받은 일봉의 시작 시각을 UTC로 쓰면?

1) 매일 00:00 UTC
2) 전날 16:00 UTC
3) 매일 08:00 UTC
4) 매일 09:00 UTC

먼저 답을 고르고 내려간다.

정답 2) 기본 1D는 홍콩 시간(UTC+8) 00:00 시작 → UTC로는 전날 16:00(한국 시간 01:00). 1분봉을 UTC 기준으로 집계한 일봉과 OKX 1D를 비교하면 값이 다르다. "버그"가 아니라 경계가 다른 것이다. UTC 일봉은 `1Dutc`.

☆ 그 밖의 선택

거래소 과거 데이터 파일	웹에서 기간별 파일을 받는 방식. 초기 백필이 빠를 수 있으나 제공 범위·형식·최신분 연결은 결국 코드로 해야 함 **[문서 확인]**
HTTP 클라이언트	`RestClient` (Spring 6.1+, 동기·간결, 기본안) · `WebClient` (리액티브, 지금은 과함) · `java.net.http.HttpClient` (의존성 0) · OkHttp
XChange 라이브러리	Java판 ccxt에 가까운 거래소 통합 라이브러리. 편하지만 이번 목표가 "HTTP·JSON·레이트 리밋을 직접 다뤄 보기"라면 직접 구현이 학습 가치가 큼
레이트 리미터	직접 구현(토큰 버킷) · Bucket4j · Resilience4j RateLimiter. 원리를 한 번 직접 짜 보고 라이브러리로 교체하는 순서 추천
재시도	네트워크 오류·429·5xx만 재시도, 지수 백오프(1s → 2s → 4s …) + 최대 횟수. 4xx(파라미터 오류)는 재시도해도 소용없음
중단 후 재개	진행 상태를 메모리가 아니라 **DB의 min(ts)/max(ts)**에서 다시 계산 → 상태 파일이 필요 없음

6. 주기적 업데이트 실행 방식 ★ 기준 시점이 핵심

Spring @Scheduled	앱 안에서 주기 실행. 설정 한 줄. 앱이 켜져 있어야 함. 기본안
Quartz	잡 상태를 DB에 저장, 클러스터·미실행 보정 지원. 지금은 과함
외부 cron / launchd	OS가 정해진 시각에 실행 명령을 호출. 앱을 상시 띄울 필요 없음
수동 실행 커맨드	`./gradlew bootRun --args='--sync'` 처럼 필요할 때 실행. 개발 초기엔 가장 단순

생각해 보기 "매일 어제 하루치를 받는다"와 "DB의 마지막 확정 ts 이후를 받는다" 중 노트북을 사흘 꺼 뒀을 때 안전한 쪽은?

먼저 답을 생각해 보고 아래로 내려간다.

후자. "어제 하루치"는 꺼져 있던 사흘 중 이틀이 영영 비게 된다. "마지막 ts 이후"는 언제 켜든 빈 만큼 자동으로 따라잡는다. 실행 시점이 아니라 **데이터 상태를 기준**으로 할 일을 계산하는 것, 이것이 증분 수집 설계의 핵심이다.

중복 실행	같은 작업이 동시에 두 번 돌아도 upsert라 데이터는 안전. 다만 호출 제한을 두 배로 쓰므로 단일 실행을 보장하는 편이 좋다 → 단일 앱이면 스케줄러 스레드 1개로 충분, 여러 인스턴스면 DB 락(`GET_LOCK`)이나 ShedLock
미확정 마지막 봉	증분 조회 결과 맨 앞 봉은 진행 중일 수 있음 → confirm = 1만 저장하면 다음 실행 때 자연히 채워짐

확인 9. Spring @Scheduled에서 "이전 실행이 끝난 시점부터 일정 시간 뒤에 다음 실행"을 의미하는 속성은?

1) fixedRate
2) fixedDelay
3) initialDelay
4) cron

먼저 답을 고르고 내려간다.

정답 2) fixedRate는 "시작 시점 기준" 간격이라 작업이 간격보다 길어지면 밀린 실행이 곧바로 이어 붙는다. 수집처럼 실행 시간이 들쭉날쭉한 작업은 fixedDelay가 안전하다. initialDelay는 앱 시작 후 첫 실행까지의 대기 시간이다.

7. 데이터 정합성 ★ 백테스트 신뢰도를 결정

★ 누락(gap) 탐지

1분봉은 ts가 정확히 60,000ms씩 증가해야 한다. 이웃한 두 봉의 간격이 60,000보다 크면 그 사이가 비어 있다.

```
SELECT inst_id, prev_ts, ts,
       (ts - prev_ts) DIV 60000 - 1 AS missing
FROM (
  SELECT inst_id, ts,
         LAG(ts) OVER (PARTITION BY inst_id ORDER BY ts) AS prev_ts
  FROM candle_1m
  WHERE inst_id = 'BTC-USDT-SWAP'
) t
WHERE ts - prev_ts > 60000;
```

간단 점검: 기대 개수 = (끝 ts − 시작 ts) ÷ 60,000 + 1, 이것과 `COUNT(*)` 를 비교. 다르면 위 쿼리로 위치를 찾는다.

확인 10. 이웃한 두 1분봉의 ts 차이가 420,000ms였다. 사이에 빠진 봉은 몇 개인가?

1) 5개
2) 6개
3) 7개
4) 420개

먼저 답을 고르고 내려간다.

정답 2) 420,000 ÷ 60,000 = 7칸 차이 → 10:00과 10:07 사이에는 10:01~10:06의 6개가 빠졌다. 칸 수와 빠진 개수를 혼동해 7을 고르는 실수가 흔하다(울타리 기둥 문제).

★ "진짜 빈 구간"과 "내가 못 받은 구간"

내가 못 받은 구간	네트워크 오류, 앱 중단, 페이지네이션 버그. 다시 요청하면 채워진다
진짜 빈 구간	거래소 점검, 상장 전, 거래가 정지된 시간. 다시 요청해도 비어 있다
처리 규칙	공백 발견 → 해당 구간 재요청 → 그래도 비면 `known_gaps` (inst_id, from_ts, to_ts, reason) 테이블에 기록 → 다음 검사에서 제외. 무한 재요청을 막는 장치

★ 값 검증 규칙

- high ≥ max(open, close), low ≤ min(open, close), high ≥ low
- 가격 > 0, 거래량 ≥ 0
- ts % 60,000 = 0 (분 경계에 정확히 맞는가)
- 전 봉 대비 가격 급변(예: ±20%)은 오류가 아닐 수 있으니 삭제하지 말고 **표시만** 한다

생각해 보기 급락장 한복판 30분이 비어 있는 채로 "저가가 손절선에 닿으면 청산" 전략을 백테스트하면?

먼저 답을 생각해 보고 아래로 내려간다.

손절선을 뚫은 봉이 데이터에 없으니 백테스트는 손절을 실행하지 않고, 30분 뒤 반등한 가격에서 계속 보유한 것으로 계산한다. 실제였다면 손절됐을 거래가 "버틴 수익"으로 바뀐다. 누락은 에러를 내지 않고 결과를 **조용히 좋게** 만든다. 그래서 백테스트 시작 전에 해당 기간의 누락 검사를 통과했는지 확인하는 단계를 둔다.

8. 상위 시간봉 집계 ★ 경계와 first/last

즉석 집계	조회할 때마다 1분봉을 GROUP BY. 원본 하나, 저장 공간 추가 없음. 이 규모에선 충분히 빠름. 기본안
집계 테이블	1H, 1D를 미리 계산해 저장. 조회는 빠르지만 원본이 바뀌면 다시 계산해야 하고 둘이 어긋날 수 있음
DB 자동 집계	TimescaleDB continuous aggregate 등. DB가 갱신을 관리. 해당 DB를 쓸 때만

★ 집계 규칙

버킷 시작 = **ts DIV 버킷길이 × 버킷길이** (1H = 3,600,000 · 4H = 14,400,000 · 1D = 86,400,000)
open = 버킷 안 **첫** 봉의 open · close = **마지막** 봉의 close · high = MAX(high) · low = MIN(low) · vol = SUM(vol)

```
SELECT b.bucket, o.open, b.high, b.low, c.close, b.vol, b.cnt
FROM (
  SELECT ts DIV 3600000 * 3600000 AS bucket,
         MIN(ts) AS first_ts, MAX(ts) AS last_ts,
         MAX(high) AS high, MIN(low) AS low, SUM(vol) AS vol, COUNT(*) AS cnt
  FROM candle_1m
  WHERE inst_id = ? AND ts >= ? AND ts < ?
  GROUP BY bucket
) b
JOIN candle_1m o ON o.inst_id = ? AND o.ts = b.first_ts
JOIN candle_1m c ON c.inst_id = ? AND c.ts = b.last_ts
ORDER BY b.bucket;
```

MIN/MAX는 집계 함수라 "첫 봉의 open"을 직접 못 고른다. 그래서 첫·마지막 ts를 구한 뒤 원본에 다시 조인한다. 윈도 함수 `FIRST_VALUE`로도 풀 수 있으니 두 방식을 비교해 본다.

생각해 보기 1시간 버킷의 cnt가 58이라면 이 1시간봉을 백테스트에 써도 될까?

먼저 답을 생각해 보고 아래로 내려간다.

빠진 2분에 그 시간의 최고가나 최저가가 있었다면 high/low가 틀린 값이다. open/close도 첫·마지막 분이 빠졌다면 틀린다. cnt = 60인 버킷만 "완전"으로 표시하고, known_gaps에 기록된 진짜 공백이면 그 사실을 함께 보여 주는 것이 정직한 처리다.

확인 11. ts가 2026-01-01 05:30 UTC인 1분봉은 4시간봉 버킷 기준으로 어느 봉에 속하나?

1) 00:00 UTC 봉
2) 04:00 UTC 봉
3) 05:00 UTC 봉
4) 08:00 UTC 봉

먼저 답을 고르고 내려간다.

정답 2) 4시간 버킷은 00, 04, 08, 12, 16, 20시 시작. 05:30은 04:00~07:59 구간이다. 버킷은 항상 "시작 시각"으로 이름 붙인다는 1장의 ts 정의와 같은 규칙이다.

확인 12. 주봉을 ts DIV 604,800,000 × 604,800,000으로 만들면 한 주가 무슨 요일 00:00 UTC에 시작하나?

1) 월요일
2) 목요일
3) 일요일
4) 토요일

먼저 답을 고르고 내려간다.

정답 2) epoch 0인 1970-01-01이 목요일이라 단순 나눗셈은 목요일 경계가 된다. 월요일 시작 주봉을 원하면 4일(345,600,000ms)만큼 보정해야 한다: `(ts - 345600000) DIV 604800000 * 604800000 + 345600000` . 시간·일·4시간봉에서는 드러나지 않다가 주봉에서만 터지는 함정이다.

9. 프로젝트 기반 기술 ☆ 기본안으로 시작

항목	선택지	기본안과 이유
프레임워크	순수 Java · Spring Boot	Spring Boot. 순수 Java가 원리 학습엔 좋지만 DI·설정·스케줄러·JDBC를 직접 엮느라 시간이 들고, 목표 직무와의 연결이 약함
빌드	Gradle · Maven	Gradle(Kotlin DSL 또는 Groovy). Maven은 금융권 레거시에서 자주 보이므로 읽을 줄만 알면 됨
Java 버전	17 · 21 · 25 (모두 LTS)	21 이상. record, 패턴 매칭, 가상 스레드 사용 가능. 사용하는 Spring Boot 버전의 지원 범위 [문서 확인]
스키마 관리	Flyway · Liquibase · 수동 SQL	Flyway. `V1__create_candle.sql` 처럼 SQL 파일 그대로 버전 관리 → 배운 SQL이 그대로 쓰임
테스트	JUnit 5 · Testcontainers · MockWebServer/WireMock	매핑·커서 로직은 단위 테스트, upsert·누락 쿼리는 Testcontainers로 실제 MariaDB에서, OKX 응답은 목 서버로 고정
설정	application.yml · 프로필	local / test 프로필 분리. 종목 목록, 호출 제한, 시작 시점을 설정으로 빼기
실행	Docker Compose	MariaDB만 컨테이너로 시작, 앱은 IDE에서 실행 → 익숙해지면 앱도 컨테이너화

생각해 보기 upsert와 누락 쿼리 테스트를 H2(인메모리 DB)로 하면 안 될까?

먼저 답을 생각해 보고 아래로 내려간다.

H2는 `ON DUPLICATE KEY UPDATE` , `DIV` , 윈도 함수 등의 지원·동작이 MariaDB와 다를 수 있어 "테스트는 통과하는데 실제 DB에선 실패"가 생긴다. 이 프로젝트의 핵심 로직이 바로 DB 방언에 의존하는 SQL이므로, Testcontainers로 실제 MariaDB를 띄워 테스트하는 것이 맞다.

10. 백테스팅 엔진과의 연결 ☆ 나중에 바꿔도 됨

DB 직접 조회	가장 단순. 백테스터가 DB 스키마에 묶임
조회 API	`GET /candles?instId=&bar=&from=&to=` . 집계·완전성 검사를 한 곳에서 보장. PrismView 차트도 같은 API 사용. 기본안
파일 내보내기	기간별 Parquet/CSV로 export. 백테스터를 Python(pandas/polars)으로 만들 때 가장 빠르고, "같은 입력 파일 → 같은 결과" 재현성이 좋음

조회 API 응답에 **완전성 정보**(구성 봉 개수, known_gaps 여부)를 함께 실어 보내면, 백테스터가 불완전 구간을 스스로 걸러낼 수 있다.

11. 나중의 확장 경로 ☆ 지금 넣지 않는 이유까지

실시간 WebSocket	OKX business 채널의 캔들 구독. 추가되는 것: ping 유지, 재연결 + 재연결 직후 REST로 공백 메우기. 지금 구조(자연키 upsert, 마지막 ts 기준 증분)를 그대로 재사용
Kafka	수집과 처리(저장, 지표 계산, 알림)를 여러 소비자로 나눌 때. 소비자가 하나뿐인 지금은 운영 부담만 늘어남
Redis	미확정 최신 봉을 빠르게 공유하거나 조회 캐시가 필요할 때. 확정 봉만 다루는 1차 범위엔 불필요
점검 질문	"DB를 PostgreSQL로 바꾸면 무엇을 고쳐야 하나?" → upsert SQL과 일부 함수. 저장소 계층(Repository)에 SQL을 모아 두면 영향 범위가 한 곳으로 줄어든다

12. 함정 모음과 결정 정리 구현 시작 전에 다시 보기

틀린 생각	올바른 내용
시계열 데이터니까 시계열 DB가 필요하다	연 100만 행 규모면 RDB로 충분. 기준은 규모·접근 패턴
가격은 double이면 된다	DECIMAL ↔ BigDecimal, 문자열로 생성, 비교는 compareTo
ts는 봉이 끝나는 시각	OKX ts는 봉 **시작** 시각, UTC epoch ms
시간은 DATETIME/TIMESTAMP로	BIGINT(epoch ms) 저장, 표시할 때 변환
AUTO_INCREMENT id가 기본	자연키 (inst_id, ts) → 중복 원천 차단, 멱등
SWAP의 vol은 BTC 수량	vol은 계약 수, BTC 수량은 volCcy, USDT 금액은 volCcyQuote
candles로 과거를 다 받는다	candles는 최신 1,440개까지, 과거는 history-candles
다음 after = 응답 첫 원소 ts	응답이 최신순이라 마지막(가장 오래된) 원소 ts
OKX 1D = UTC 일봉	기본 1D는 홍콩 시간 기준, UTC는 1Dutc
JPA saveAll이면 대량 insert도 충분	IDENTITY·merge SELECT·1차 캐시 → JDBC 배치 upsert
MariaDB 드라이버도 rewriteBatchedStatements	Connector/J 3.x에서 제거, bulk 방식 기본
"어제 하루치"를 매일 받는다	DB의 마지막 확정 ts 이후를 받는다 → 자동 따라잡기
스케줄은 fixedRate	실행 시간이 들쭉날쭉하면 fixedDelay
ts 차이 7칸 = 7개 누락	칸 수 − 1 = 6개 누락
재요청해도 비면 버그	점검·상장 전일 수 있음 → known_gaps 기록
누락은 에러가 난다	에러 없이 백테스트 결과를 조용히 왜곡
ts DIV 1주 = 월요일 시작	epoch 기준 목요일 시작 → 4일 보정
테스트는 H2로	방언 의존 SQL이므로 Testcontainers 실제 MariaDB

결정 요약표 — 빈칸은 내가 채운다

항목	노트 기본안	내 결정	이유	다시 볼 조건
DBMS	MariaDB			
PK	(inst_id, ts)			
가격·거래량	DECIMAL / BigDecimal			
시간	BIGINT epoch ms (UTC)			

항목	노트 기본안	내 결정	이유	다시 볼 조건
저장 단위	1분봉만, 확정 봉만			
DB 접근	JdbcTemplate 배치 upsert			
HTTP	RestClient			
호출 제한	직접 구현 → 라이브러리			
실행	@Scheduled(fixedDelay)			
누락 처리	LAG 탐지 + known_gaps			
상위 시간봉	즉석 SQL 집계, UTC			
기반	Boot · Gradle · Java 21+			
마이그레이션	Flyway			
백테스터 연결	조회 API (+ Parquet 선택)			

ADR 한 장 쓰는 법 — 예시

> **ADR-001 · 저장소로 MariaDB 사용** · 상태: 채택 · 날짜: _____
> **배경** BTC·ETH 1분봉 연 약 105만 행, append-mostly, 종목+시간 범위 조회와 시간봉 집계가 주 용도
> **검토한 대안** PostgreSQL, TimescaleDB, ClickHouse, DuckDB+Parquet
> **결정** MariaDB + 자연키 (inst_id, ts) + upsert
> **이유** 규모상 성능 차이 무의미 · upsert·PK·윈도 함수로 정합성 확보 용이 · 이미 익숙해 Java 학습에 집중 가능
> **결과·대가** 시간 버킷·연속 집계를 SQL로 직접 작성해야 함
> **다시 검토할 조건** 종목 수백 개 이상, 틱 데이터 도입, 집계 조회가 느려질 때

면접 답변 구조 — "왜 이 DB를 골랐나요?"

1. **결론** — "MariaDB를 썼습니다."
2. **데이터 특성** — "두 종목 1분봉이라 연 100만 행 정도이고, 거의 추가만 일어납니다."
3. **비교 기준** — "이 규모에선 성능 차이가 의미 없어서, 중복 방지·누락 검사 같은 정합성 도구와 학습 집중도를 기준으로 봤습니다."
4. **대가** — "시계열 함수가 없어 집계 SQL을 직접 작성했고, 주봉 경계 같은 문제를 겪었습니다."
5. **바꿀 조건** — "종목이 수백 개로 늘거나 틱 데이터를 다루면 TimescaleDB 같은 시계열 DB를 검토하겠습니다."

끝내기 전 체크 — 안 보고 말로 설명해 보고 체크

☐ 2종목 1분봉 1년·5년의 행 수와 대략적 용량을 계산할 수 있다
☐ 행 기반과 열 기반의 차이, 이 프로젝트에서 성능이 기준이 아닌 이유를 말할 수 있다
☐ 자연키와 멱등성의 관계를 설명할 수 있다
☐ DOUBLE을 쓰면 안 되는 이유와 BigDecimal 생성·비교 규칙을 말할 수 있다
☐ OKX vol·volCcy·volCcyQuote의 차이와 confirm의 의미를 말할 수 있다
☐ history-candles 페이지네이션의 커서 규칙과 1년 백필 소요 시간을 계산할 수 있다
☐ OKX 1D와 UTC 일봉이 다른 이유를 설명할 수 있다
☐ JPA 대량 insert가 느린 이유 3가지를 말할 수 있다
☐ ts 차이로 누락 개수를 계산하고, 진짜 공백과 수집 실패를 구분하는 규칙을 말할 수 있다
☐ 시간봉 집계 SQL에서 open/close를 고르는 방법과 주봉 경계 함정을 설명할 수 있다
☐ 결정 요약표의 "내 결정" 칸을 모두 채웠다

확인 문제 12개 점수: ___ / 12 · 다 맞고 결정 요약표를 채웠으면 1단계(프로젝트 뼈대)로 넘어간다.

참고 자료

- OKX API v5 문서 — Market Data: Get candlesticks / Get candlesticks history, Public Data: Get instruments (okx.com/docs-v5)
- MariaDB Connector/J 3.2.0 Release Notes — bulk batching 기본 동작 (mariadb.com)
- MariaDB Jira CONJ-1077 — 3.x에서 rewriteBatchedStatements 제거 관련 (jira.mariadb.org)
- MariaDB Knowledge Base — INSERT … ON DUPLICATE KEY UPDATE, Window Functions, DECIMAL
- PostgreSQL 문서 — INSERT … ON CONFLICT · TimescaleDB 문서 — hypertable, time_bucket, continuous aggregates
- Spring Framework 문서 — JdbcTemplate batch operations, @Scheduled, RestClient

[문서 확인] 표시 항목(호출 제한, 과거 데이터 범위, 라이선스, 버전별 지원)은 바뀔 수 있으므로 구현 직전에 공식 문서로 다시 확인한다.